Rayat Shikshan Sanstha's
KARMAVEER BHAURAO PATIL COLLEGE,VASHI
(Empowered Autonomous)

Reaccredited NAAC with Grade 'A++' (CGPA3.51)|ISO 9001:2015
Certified Institute 'Best College' Award by University of Mumbai

[DEPARTMENT OF INFORMATION TECHNOLOGY]

CERTIFICATE

This is to certify that **Mr./Miss: _____**student of class from **Karmaveer Bhaurao Patil College, Vashi, Navi Mumbai** has satisfactorily completed the practical course in subject **Microservices Architecture.** As per the syllabus laid by the college during the Academic. Year **2025-26.**

ROLL NO.:

EXAM NO.:

Date: / /

MANOJ CHOUDHARY
Course coordinator

MADHURI GABHANE
Head, Department of IT

External Examiner

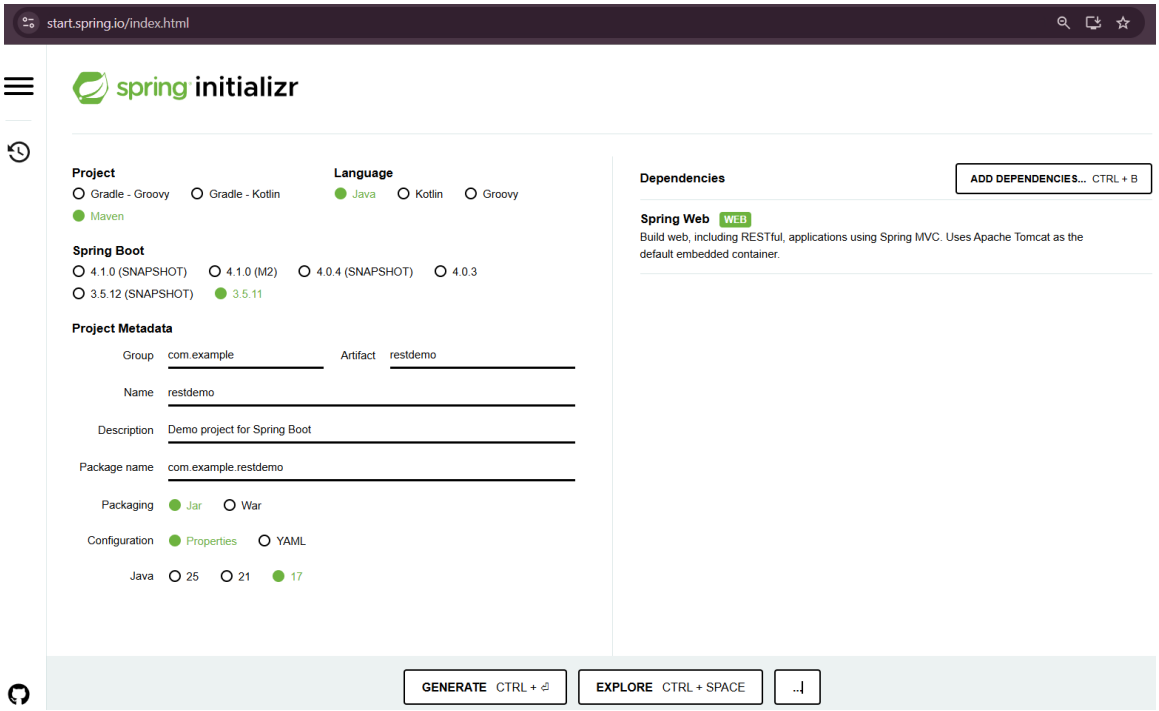
INDEX

SR.NO.	PRACTICAL NAME	DATE	SIGN
1.	Introduction to REST API using Spring Boot.		
2.	Creating a Monolithic Application using Spring boot.		
3.	Converting Monolithic Application to Microservices		
4.	Simple Login Authentication System using Microservices.		
5.	Basic Fault Tolerance (Circuit Breaker Concept)		
6.	Implementing Communication Between Microservices using REST API		
7.	Implementing Service Registry-Service Discovery using Eureka		

Practical : 1

Aim: Introduction to REST API using Spring Boot

- Software Required
  - Java 17
  - IntelliJ IDEA
  - Postman
  - Internet Browser



STEP 1 – Create Spring Boot Project

1.1 Open Spring Initializr
Open browser and go to: https://start.spring.io

1.2 Fill Project Details
Select:
Project: Maven
Language: Java
Spring Boot: 3.x
Group: com.example
Artifact: restdemo
Name: restdemo
Packaging: Jar

Java: 17

Click: Add Dependency

Select: Spring Web

Click: Generate Download the ZIP file and extract it.

STEP 2 – Open Project in IntelliJ

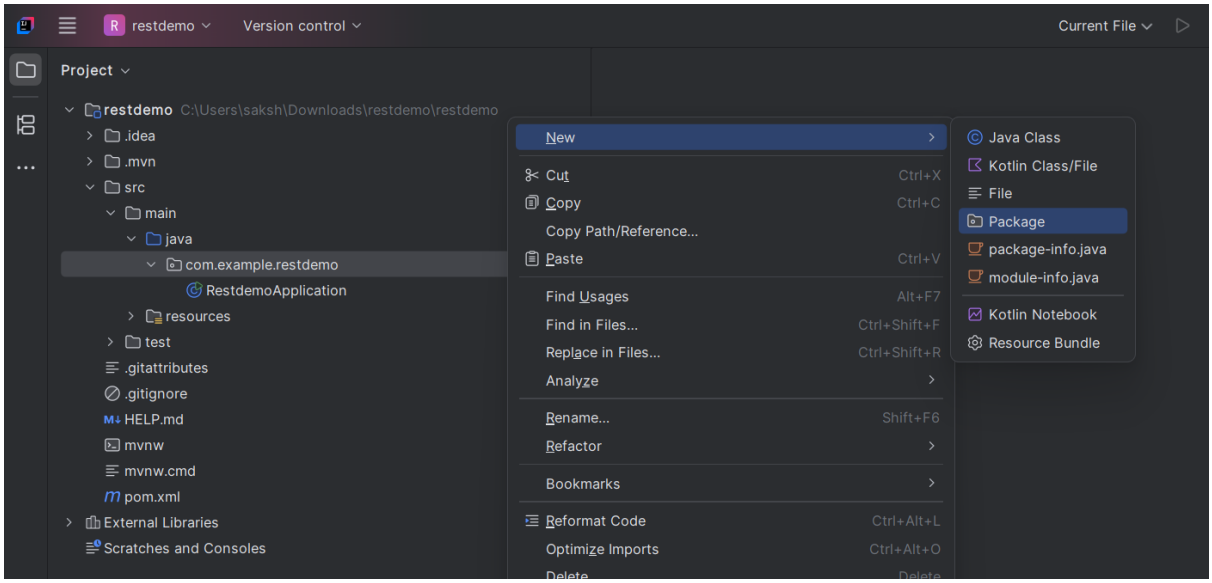
1. Open IntelliI IDEA
2. Click Open
3. Select extracred folder restdemo
4. Click OK
5. Wait for Maven dependencies to download

STEP 3 : Create Packages

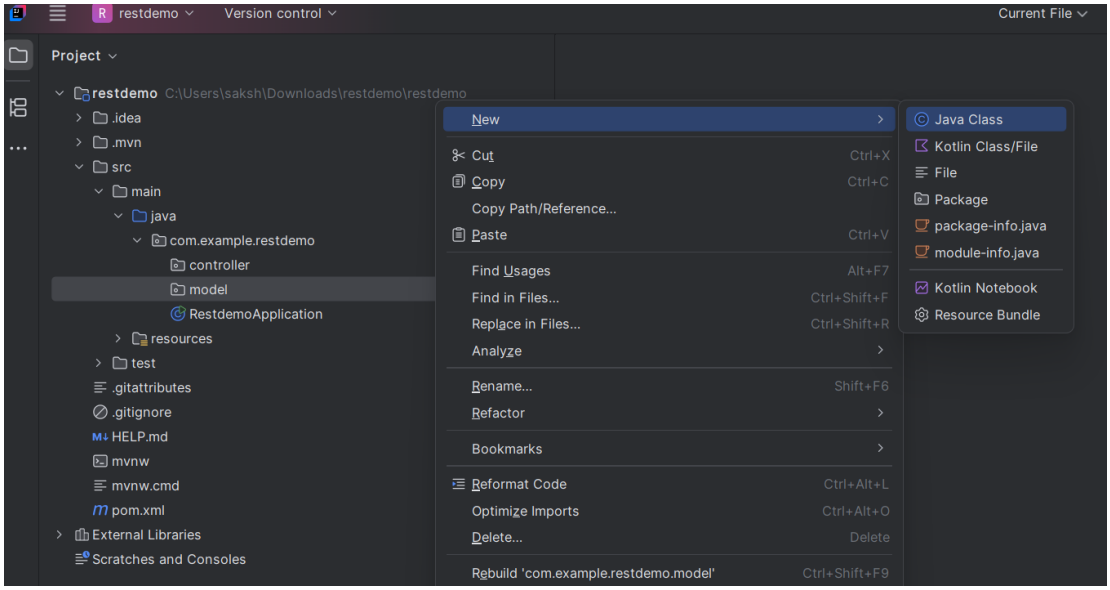
Right click: com.example.restdemo

Create: Package → model

Package → controller



STEP 4 : Create Model Class



Create **Student.java**

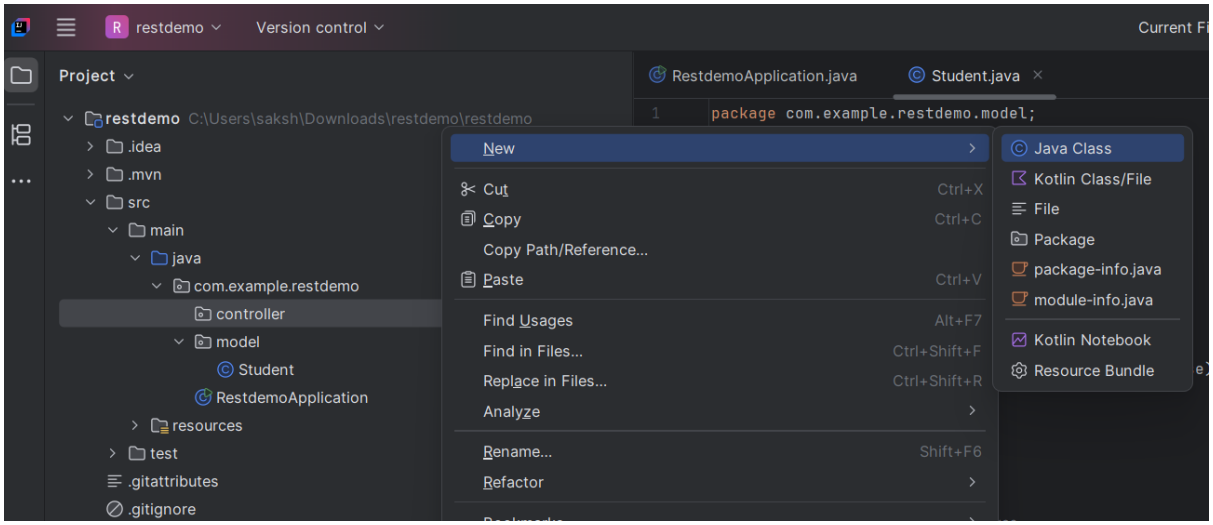
File path: src/main/java/com/example/restdemo/model/Student.java

Code :

```
package com.example.restdemo.model;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }
```

```
    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

STEP 6 : Create REST Controller



Create **StudentController.java**

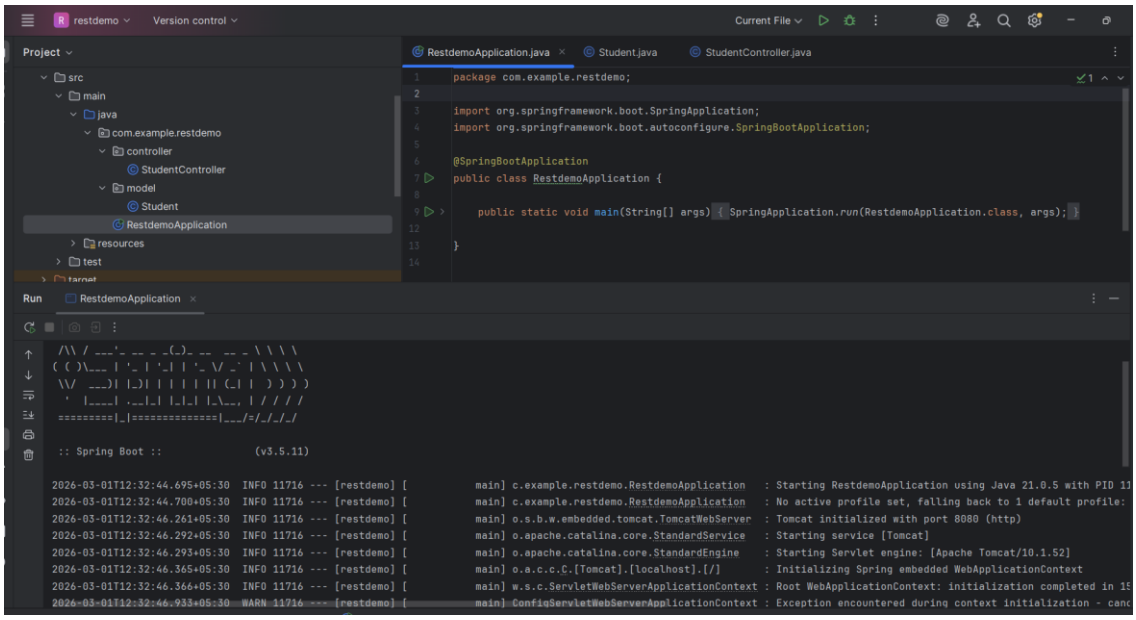
<u>FilePath:</u>

src/main/java/com/example/restdemo/controller/StudentController.java

Code:

```
package com.example.restdemo.controller;

import org.springframework.web.bind.annotation.*;

import com.example.restdemo.model.Student;


@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping
    public Student getStudent() {
        return new Student(1, "Rahul", "BSc IT");
    }

    @PostMapping
    public Student createStudent(@RequestBody Student student) {
        return student;
    }
}
```

STEP 6: Run Application

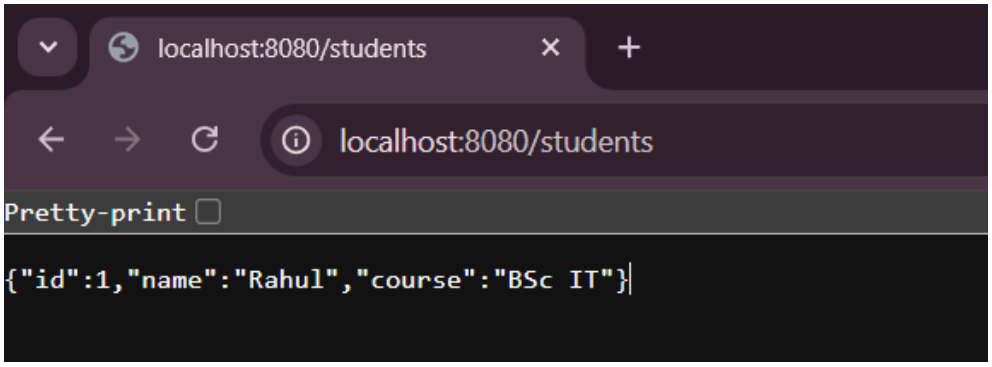


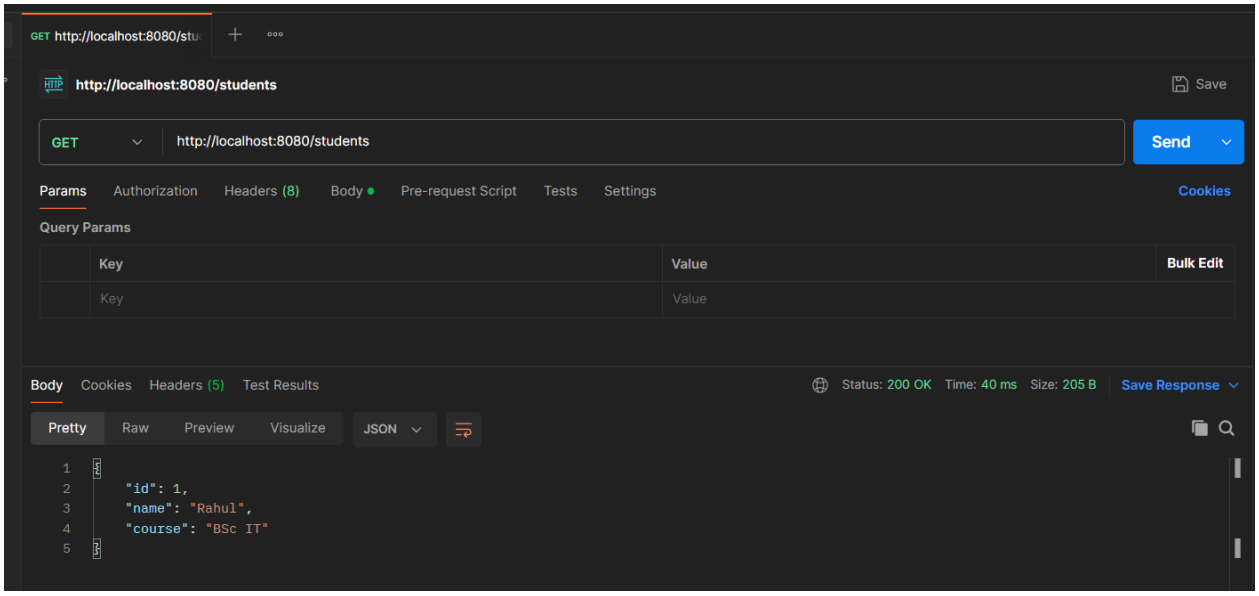
Open: RestDemoApplication.java

Right click → Run

Console should display: Tomcat started on port 8080

STEP 7: Test GET API





Open browser or Postman: http://localhost:8080/students

Response:

{

"id": 1,

"name": "Rahul",

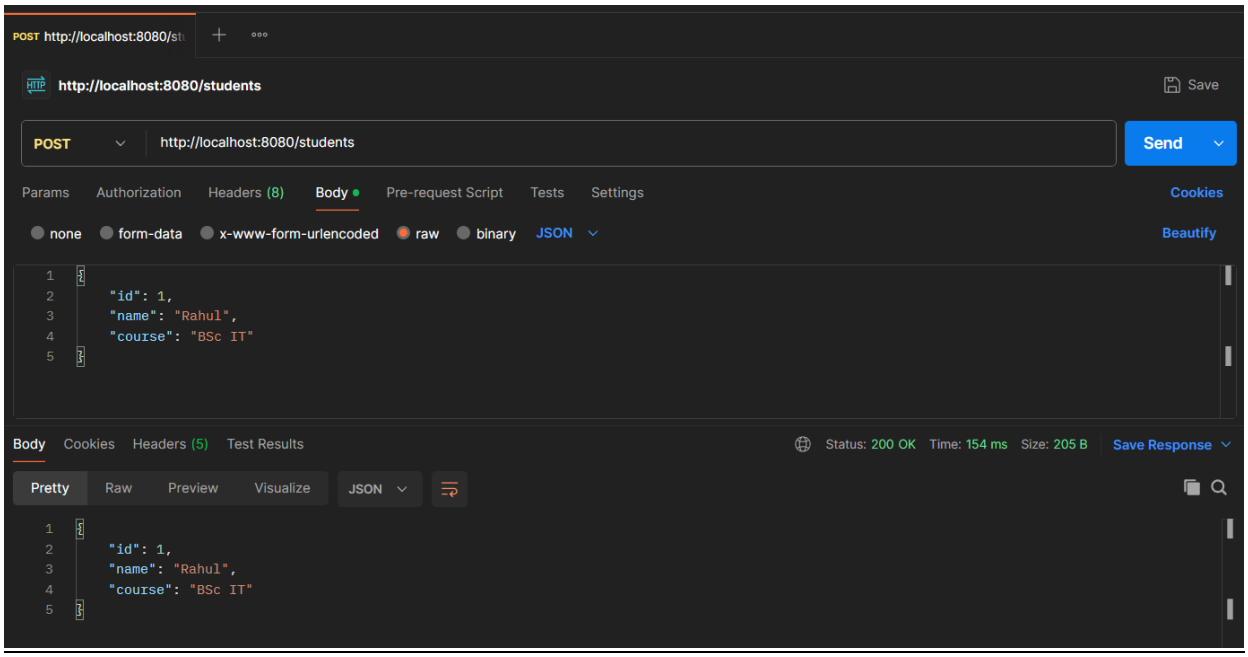
"course": "BSc IT"

}

STEP 8: Test POST API

Open Postman.

1. Select Method → POST
2. Enter URL: http://localhost:8080/students



Open Postman.

1. Select Method → POST
2. Enter URL: http://localhost:8080/students
3. Go to Body → Select raw → JSON
4. Enter: {

   "id": 2,

   "name": "Anjali",

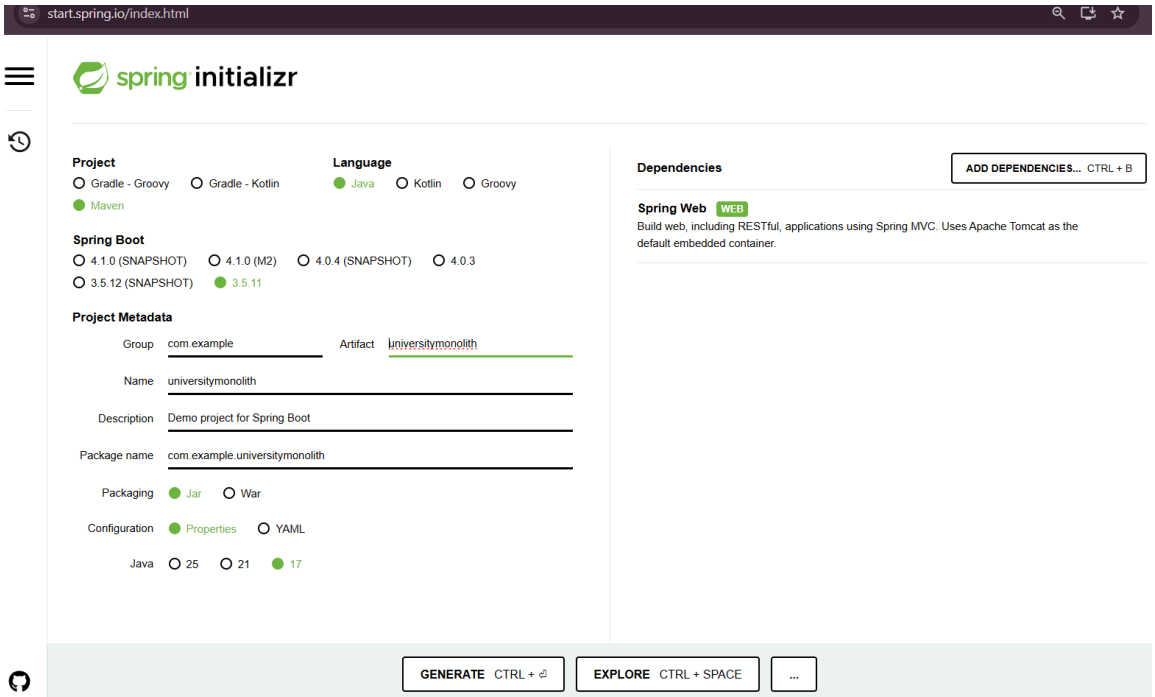
   "course": "BSc CS"

   }

Practical : 2

Aim: Creating a Monolithic Application using Spring Boot.

STEP 1:

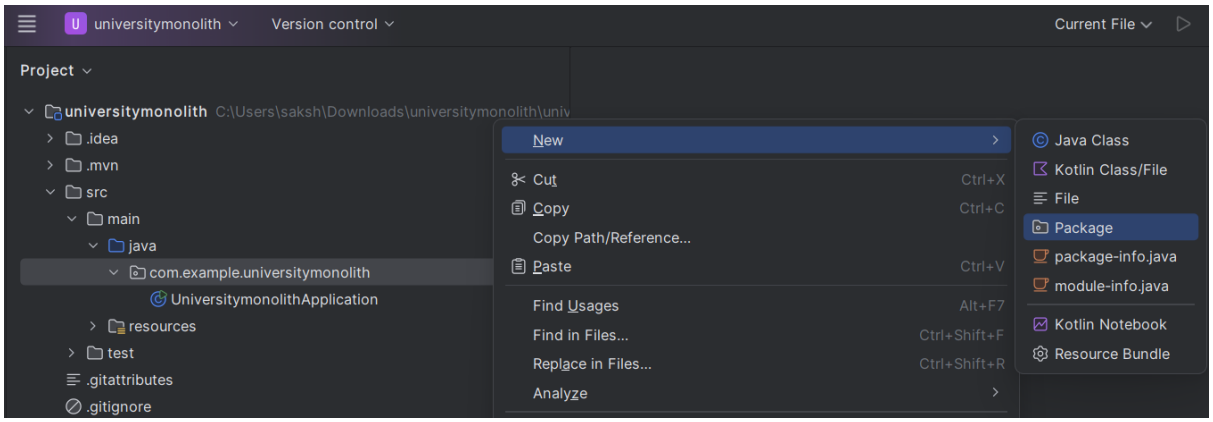


Click: Generate Download the ZIP file and extract it.

STEP 2: Open Project in IntelliJ

1. Open IntelliI IDEA
2. Click Open
3. Select extracred folder restdemo
4. Click OK

STEP 3: Create Packages



Right click: com.example.universitymonolith

- Package → model
- Package → controller

STEP 4: Create Model Classes in model package

File path: src/main/java/com/example/universitymonolith/**model**/Student.java

Create **Student.java**

```
package com.example.universitymonolith.model;

public class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
}
```

Create **Course.java**

```
package com.example.universitymonolith.model;
public class Course {
    private int id;
    private String courseName;

    public Course(int id, String courseName) {
        this.id = id;
        this.courseName = courseName;
    }
```

```
    public int getId() {
        return id;
    }

    public String getCourseName() {
        return courseName;
    }
}
```

Create **Result.java**

```
package com.example.universitymonolith.model;
public class Result {
    private int studentId;
    private int marks;

    public Result(int studentId, int marks) {
        this.studentId = studentId;
        this.marks = marks;
    }

    public int getStudentId() {
        return studentId;
    }

    public int getMarks() {
        return marks;
    }
}
```

STEP 5: Create Controller in Controller Package

File path:
src/main/java/com/example/universitymonolith/**controller**/UniversityController.

- Create **UniversityController**

```
package com.example.universitymonolith.controller;
import org.springframework.web.bind.annotation.*;
```

```
import com.example.universitymonolith.model.*;
@RestController
@RequestMapping("/university")
public class UniversityController {
    @GetMapping("/student")
    public Student getStudent() {
        return new Student(1, "Rahul");
    }

    @GetMapping("/course")
    public Course getCourse() {
        return new Course(101, "BSc IT");
    }

    @GetMapping("/result")
    public Result getResult() {
        return new Result(1, 85);
    }
}
```

STEP 6: Run Application

Open: UniversityMonolithApplication.java

Right click → Run

STEP 7: Test APIs

Using browser or Postman:

http://localhost:8080/university/student

http://localhost:8080/university/course

http://localhost:8080/university/result

All responses come from one application.

localhost:8080/university/stude

localhost:8080/university/student

Pretty-print


```
{
  "id": 1,
  "name": "Rahul"
}
```


localhost:8080/university/result

localhost:8080/university/result

Pretty-print


```
{
  "studentId": 1,
  "marks": 85
}
```


localhost:8080/university/cours

localhost:8080/university/course

Pretty-print


```
{
  "id": 101,
  "courseName": "BSc IT"
}
```

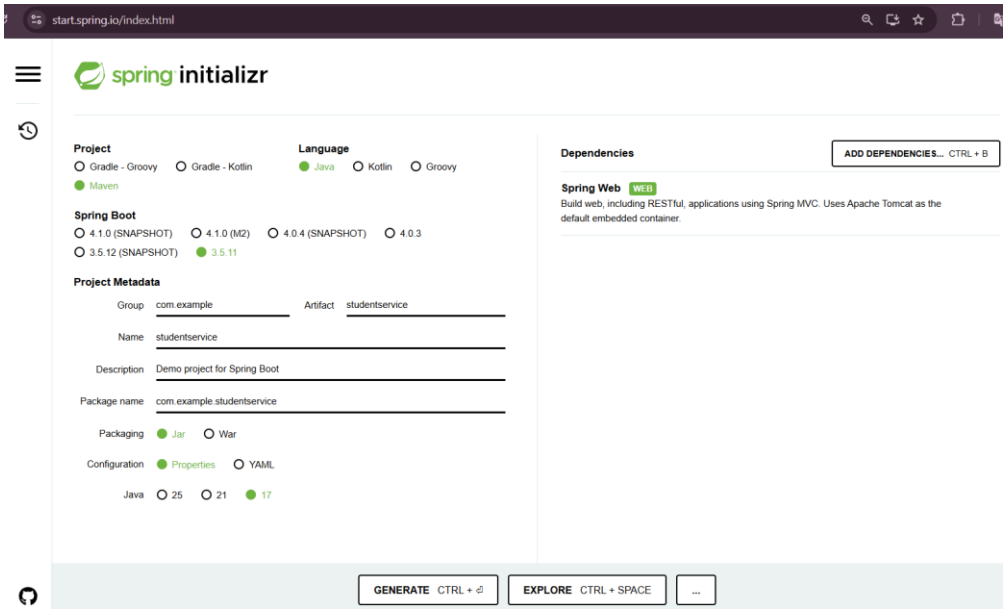
Practical: 3

Aim: Converting Monolithic Application to Microservices

Part I: Create Student Service

STEP 1: Create project using spring initializer

1. Open browser
2. Go to: https://start.spring.io

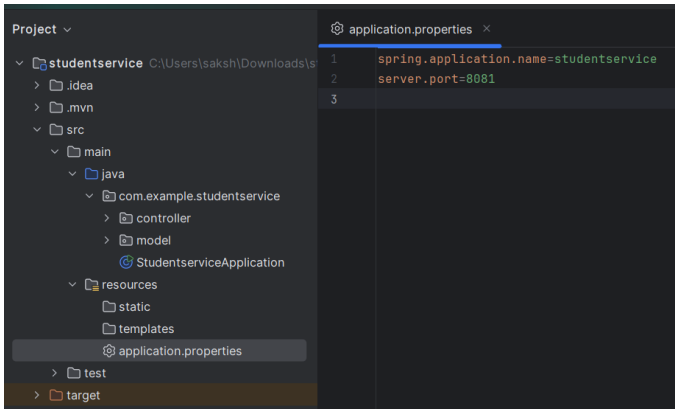


3.Click on Generate. Download ZIP file and Extract the folder

STEP 2: Open Project in IntelliJ

1. Open IntelliJ IDEA
2. Click Open
3. Select extracted folder student-service

STEP 3: Configure Port Number



Open file: src/main/resources/application.properties

Add: server.port=8081

STEP 4: Create Package

Inside: src/main/java/com/example/studentservice

Create two packages:

1. model
2. controller

STEP 5: Create Student Model in model package

Create **Student.java**

```
package com.example.studentservice.model;
public class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

STEP 6: Create Student Controller

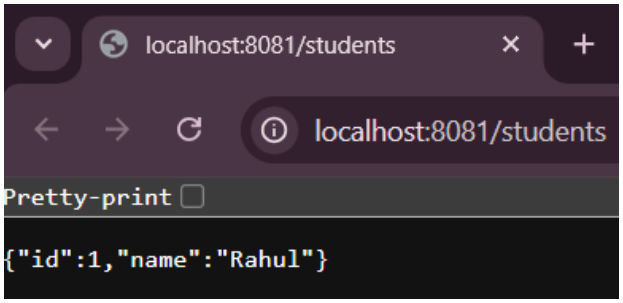
```
package com.example.studentservice.controller;
import org.springframework.web.bind.annotation.*;
```

```
import com.example.studentservice.model.Student;

@RestController
@RequestMapping("/students")
public class StudentController {
    @GetMapping
    public Student getStudent() {
        return new Student(1, "Rahul");
    }
}
```

STEP 7: Run Student Service

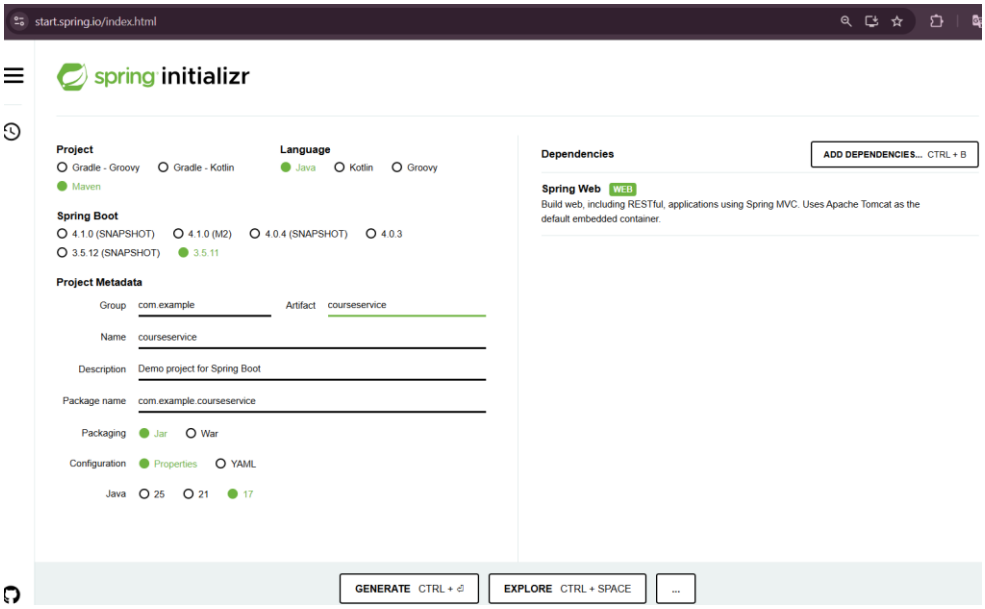
Run: StudentServiceApplication.java

Test in browser or Postman: http://localhost:8081/students



PART 2 – CREATE COURSE SERVICE

STEP 1: Create New Project



Generate and open in IntelliJ.

STEP 2: Change Port Open:

src/main/resources/application.properties

Add: server.port=8082

STEP 3: Create Packages

Inside main package create:

• model • controller

STEP 4: Create Course Model in **model package**

Create **Course.java**

```
package com.example.courseservice.model;
public class Course {
    private int id;
    private String courseName;

    public Course(int id, String courseName) {
        this.id = id;
        this.courseName = courseName;
    }

    public int getId() {
        return id;
    }

    public String getCourseName() {
        return courseName;
    }
}
```

STEP 5: Create Course Controller in **controller package**

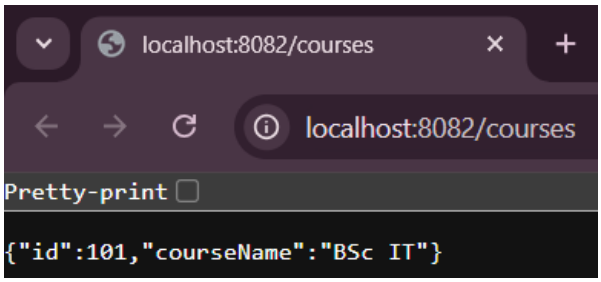
Create **CourseController.java**

```
package com.example.courseservice.controller;
import org.springframework.web.bind.annotation.*;
import com.example.courseservice.model.Course;

@RestController
@RequestMapping("/courses")
public class CourseController {
    @GetMapping
    public Course getCourse() {
        return new Course(101, "BSc IT");
    }
}
```

STEP 6: Run Course Service

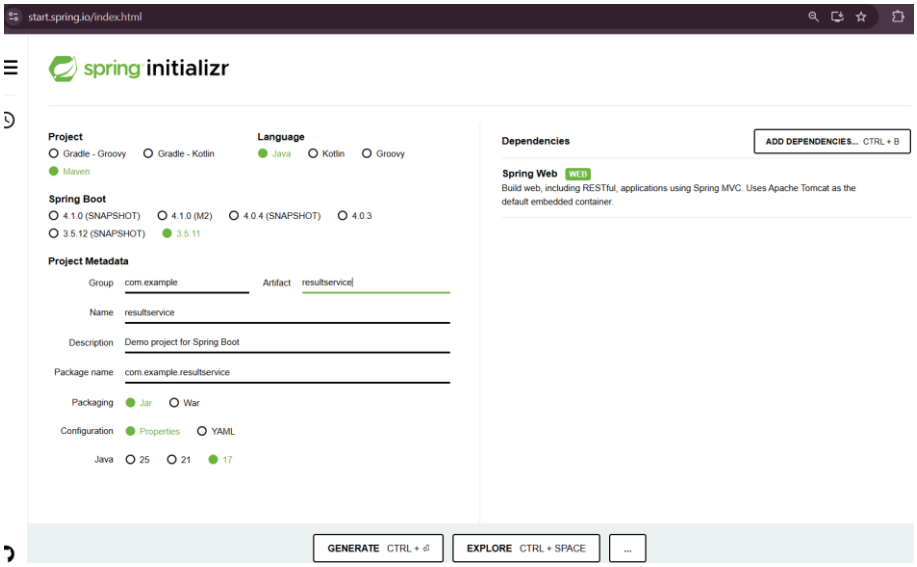
Run: CourseServiceApplication.java

Test: http://localhost:8082/courses



PART 3 – CREATE RESULT SERVICE

STEP 1: Create New Project



Generate and Open in IntelliJ.

STEP 2: Change Port

src/main/resources/application.properties

Add: server.port=8083

STEP 3: Create Packages

• model • controller

STEP 4: Create Result Model in model package

Create **Result.java**

```
package com.example.resultservice.model;
public class Result {
    private int studentId;
    private int marks;

    public Result(int studentId, int marks) {
        this.studentId = studentId;
        this.marks = marks;
    }

    public int getStudentId() {
        return studentId;
    }

    public int getMarks() {
        return marks;
    }
}
```

STEP 5: Create Result Controller in controller package

Create **ResultController.java**

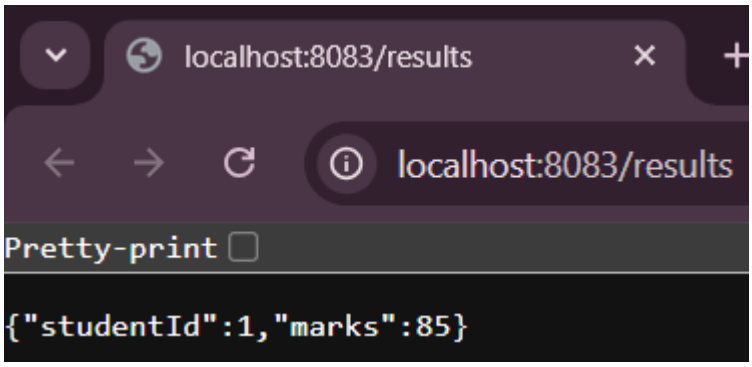
```
package com.example.resultservice.controller;
```

```
import org.springframework.web.bind.annotation.*;
import com.example.resultservice.model.Result;

@RestController
@RequestMapping("/results")
public class ResultController {
    @GetMapping
    public Result getResult() {
        return new Result(1, 85);

    }
}
```

STEP 6: Run Result Service

Run: ResultServiceApplication.java

Test: http://localhost:8083/results

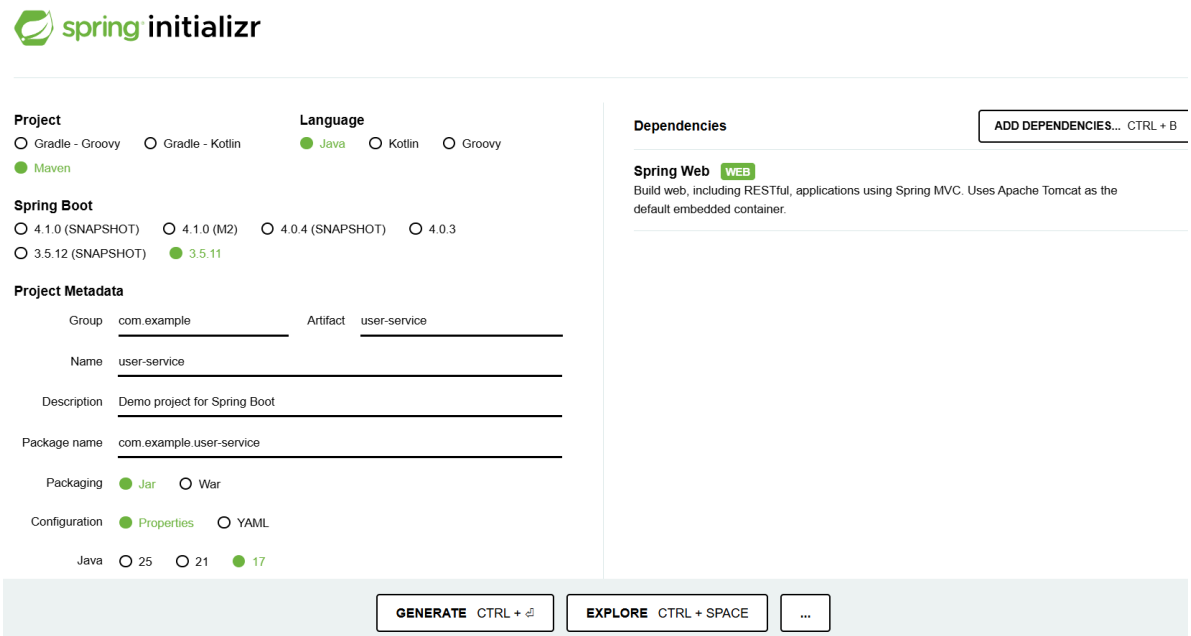


Practical : 4

Aim: Simple Login Authentication System using Microservices.

Part I: User Service – Stores user information

STEP 1: Create project using spring initializer



Click Generate and open in IDE.

Step 2: Project Structure user-service

```
user-service
 |
 |-- controller
 |       UserController.java
 |
 |-- model
 |       User.java
 |
 |-- service
 |       UserService.java
```

Step 3: Create User Model

File: model/User.java

```
package com.example.user_service.model;

public class User {
    private String username;
    private String password;
```

```
    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
    public String getUsername() {
        return username;
    }
    public String getPassword() {
        return password;
    }
}
```

STEP 4: Create User Service

File service/**UserService.java**

```
package com.example.user_service.service;

import com.example.user_service.model.User;
import org.springframework.stereotype.Service;

    @Service
    public class UserService {
        public User getUser(String username) {
            if(username.equals("admin")) {
                return new User("admin","1234");
            }
            return null;
        }
    }
```

- **<u>remember: username = admin & password = 1234</u>**

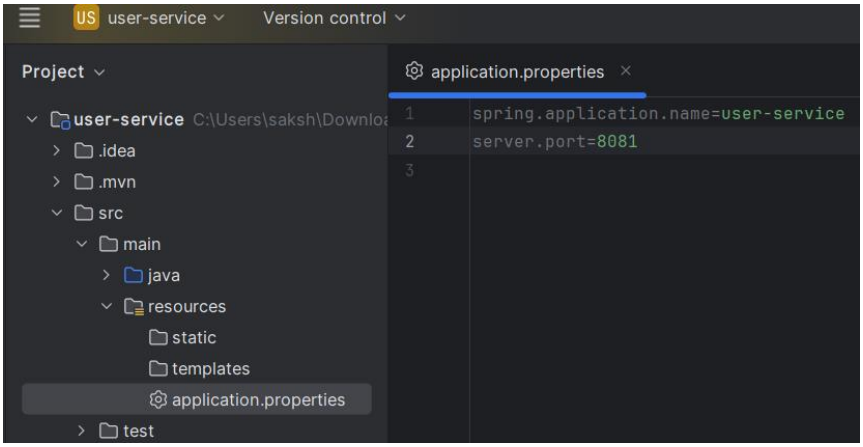
Step 5: Create User Controller

File controller/UserController.java

```
package com.example.user_service.controller;

import com.example.user_service.model.User;
import com.example.user_service.service.UserService;
import org.springframework.web.bind.annotation.*;

@RestController
```

```
@RequestMapping("/user")
public class UserController {
    private final UserService userService;
    public UserController(UserService userService) {
        this.userService = userService;
    }
    @GetMapping("/{username}")
    public User getUser(@PathVariable String username) {
        return userService.getUser(username);
    }
}
```

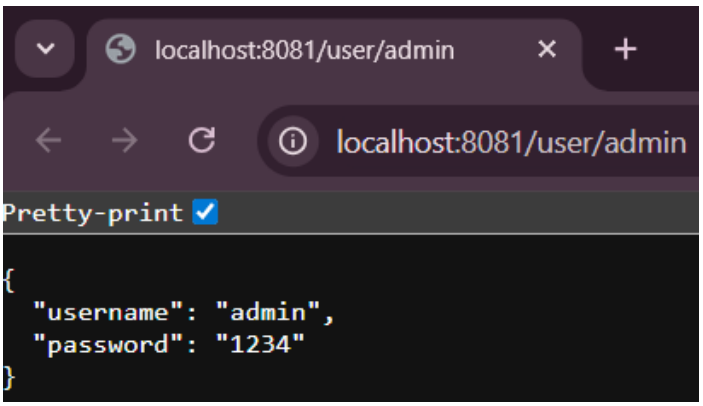
Step 6: Change Port Number



Step 7: Run User Service

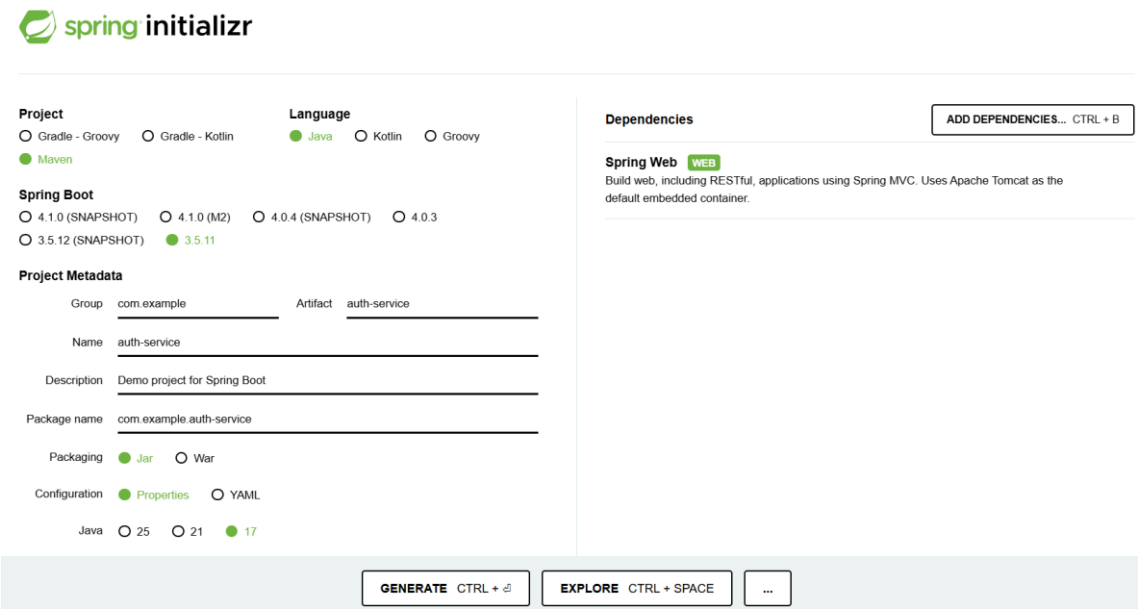
Run UserServiceApplication

Test in browser: http://localhost:8081/user/admin



Part II: Create Authentication Microservice

STEP 1: Create project using spring initializer



Click Generate and open in IDE.

STEP 2: Project Structure

```
auth-service
 |
 |-- controller
 |       AuthController.java
 |
 |-- model
 |       LoginRequest.java
 |
 |-- service
 |       AuthService.java
```

STEP 3: Create Login Request Model

File model/LoginRequest.java

```
package com.example.auth_service.model;

public class LoginRequest {
    private String username;
    private String password;
    public String getUsername() {
        return username;
    }
    public void setUsername(String username) {
```

```
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

STEP 4: Create Auth Service

File service/AuthService.java

```
package com.example.auth_service.service;

import com.example.auth_service.model.LoginRequest;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
@Service
public class AuthService {
    public String login(LoginRequest request) {
        RestTemplate restTemplate = new RestTemplate();
        String url = "http://localhost:8081/user/" + request.getUsername();
        try {
            Object user = restTemplate.getForObject(url, Object.class);
            if(user != null && request.getPassword().equals("1234")) {
                return "Login Successful";
            }
        } catch(Exception e) {
            return "User Not Found";
        }
        return "Invalid Password";
    }
}
```

STEP 5: Create Auth Controller

File controller/AuthController.java

```
package com.example.auth_service.controller;

import com.example.auth_service.model.LoginRequest;
```

```
import com.example.auth_service.service.AuthService;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/auth")
public class AuthController {
    private final AuthService authService;
    public AuthController(AuthService authService) {
        this.authService = authService;
    }
    @PostMapping("/login")
    public String login(@RequestBody LoginRequest request) {
        return authService.login(request);
    }
}
```

STEP 6: Change Port

File: application.properties

```
server.port=8082
```

STEP 7: Run Both Services

Run

- User Service , Auth Service

Both must run simultaneously.

STEP 8: Test using Postman

POST Request

http://localhost:8082/auth/login

Body → JSON

{ "username":"admin", "password":"1234" }

Output:

- Login Successful
- Wrong password:

  Invalid Password

http://localhost:8082/auth/login

Save

POST http://localhost:8082/auth/login Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "username": "admin",
    "password": "1234"
}
```

Body Cookies Headers (5) Test Results Status: 200 OK Time: 374 ms Size: 180 B Save Response

Pretty Raw Preview Visualize Text

```
Login Successful
```

http://localhost:8082/auth/login

Save

POST http://localhost:8082/auth/login Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
    "username": "admin",
    "password": "3456"
}
```

ody Cookies Headers (5) Test Results Status: 200 OK Time: 31 ms Size: 180 B Save Response

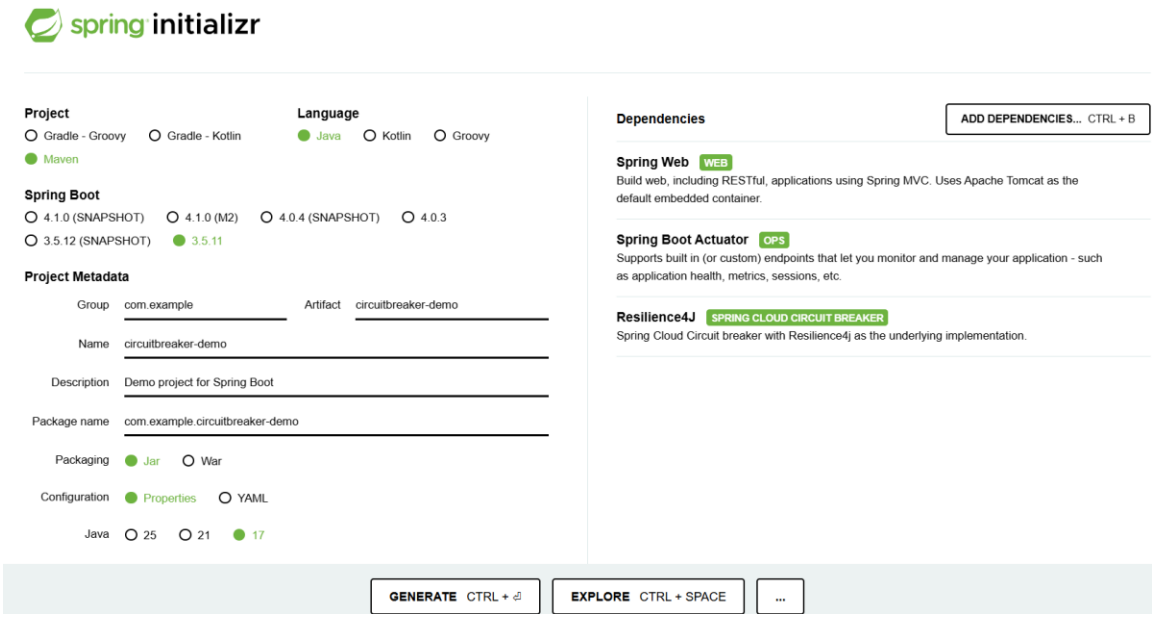
Pretty Raw Preview Visualize Text

```
Invalid Password
```

Practical : 5

Aim: Basic Fault Tolerance (Circuit Breaker Concept)

STEP 1: Create Project using Spring Initializer

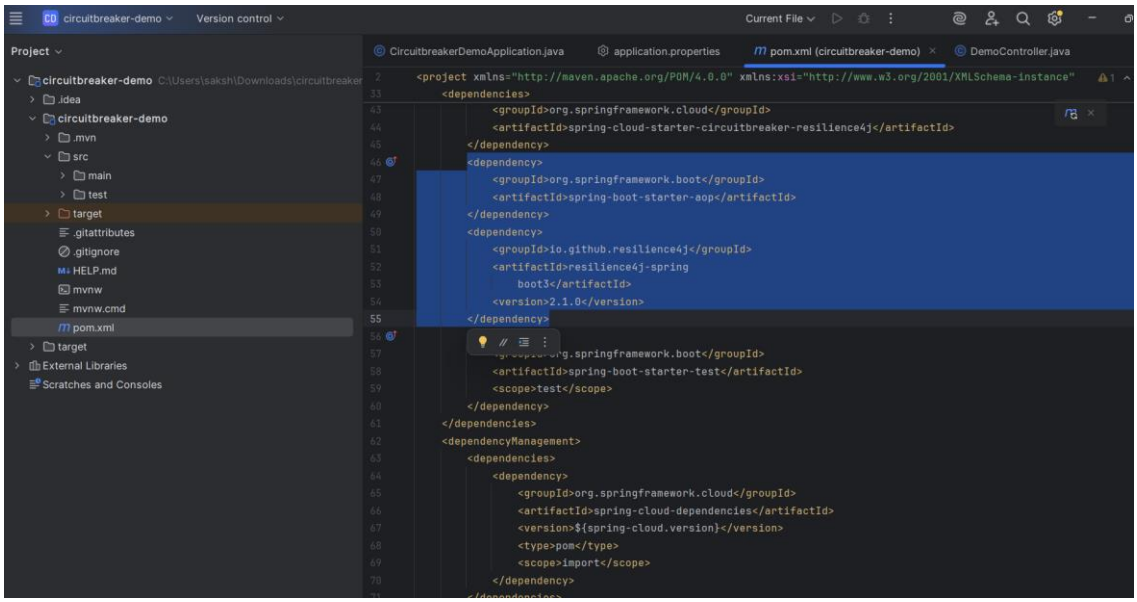


Click on generate and open in IntelliJ IDE

STEP 2: Add Resilience4j Dependency

Open : pom.xml

Inside the <dependencies> section add:




```
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring
      boot3</artifactId>
    <version>2.1.0</version>
</dependency>
```

STEP 3: Create Controller Package

Right click: com.example.circuitbreaker_demo

Select: New → Package

Name the package: controller

STEP 4: Create Controller Class

Right click on controller package

Select: New → Java Class

Name the class: DemoController

STEP 5: Write Controller Code

Add the following code in **DemoController.java**

```
package com.example.circuitbreaker_demo.controller;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class DemoController {
    private int count = 0;
    @GetMapping("/service")
    @CircuitBreaker(name = "demoService", fallbackMethod =
          "fallbackResponse")
    public String callService() {
        count++;
        if (count % 2 == 0) {
            throw new RuntimeException("Service Failed");
```

```
        }
        return "Service Response Successful";
    }
    public String fallbackResponse(Exception e) {
        return "Fallback Response: Service is temporarily unavailable";
    }
}
```

STEP 6: Configure Circuit Breaker

Open the file:

src/main/resources/application.properties

Add the following configuration:

```
spring.application.name=circuitbreaker-demo
server.port=8081
resilience4j.circuitbreaker.instances.demoService.registerHealthIndicator=true
resilience4j.circuitbreaker.instances.demoService.failureRateThreshold=50
resilience4j.circuitbreaker.instances.demoService.slidingWindowSize=5
resilience4j.circuitbreaker.instances.demoService.waitDurationInOpenState=10s
resilience4j.circuitbreaker.instances.demoService.permittedNumberOfCallsInHa
lfOpenState=3
```

STEP 7: Run the Application Open:

Open:

CircuitbreakerDemoApplication.java(Main running file)

```
package com.example.circuitbreaker_demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.context.annotation.EnableAspectJAutoProxy;
@SpringBootApplication
@EnableAspectJAutoProxy
public class CircuitbreakerDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(CircuitbreakerDemoApplication.class,
            args);
    }
}
```

Click on run button

Console output should show:

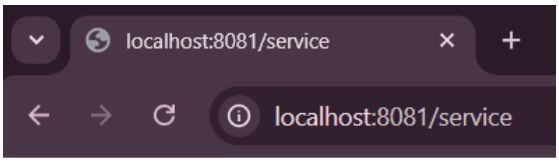
Tomcat started on port 8081

STEP 8: Test the Application

Open the browser and enter: http://localhost:8081/service

STEP 9: First Request

Output: Service Response Successful



Service Response Successful

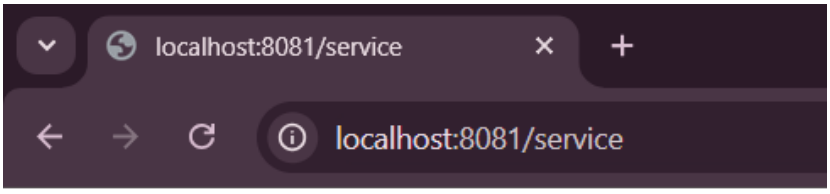
This means the service is working normally.

STEP 10: Simulate Failure

Refresh the browser page.

Output:

Fallback Response: Service is temporarily unavailable



Fallback Response: Service is temporarily unavailable

This shows that the Circuit Breaker activated the fallback method.

STEP 11: Demonstration Flow Requests will behave as follows:

Request 1 → Service Response Successful

Request 2 → Fallback Response

Request 3 → Service Response Successful

Request 4 → Fallback Response

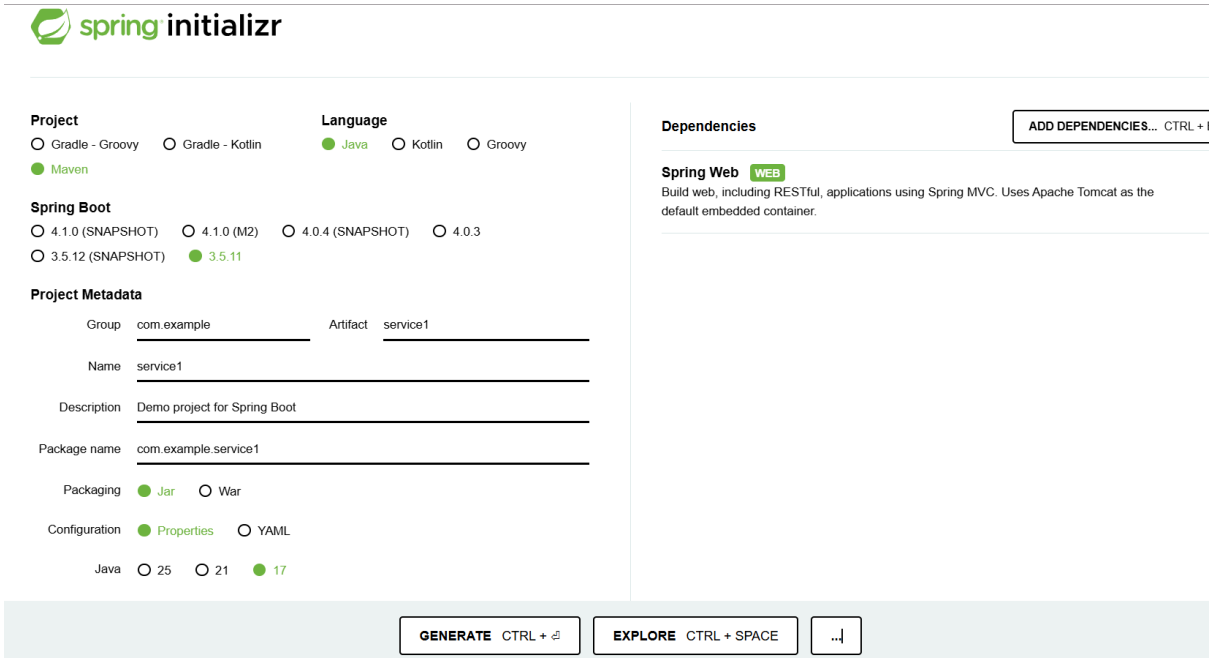
Practical : 6

Aim: Implementing Communication Between Microservices using REST API

Part I

STEP 1: Create Service 1 using Spring Initializer

Open browser and go to: https://start.spring.io



Click on generate.Download and extract the project. Open it in IntelliJ IDEA.

STEP 2: Create Controller in Service1

Right click: com.example.service1

Select: New → Package

Name the package: controller

STEP 3: Create Controller Class

Right click on controller package

Select: New → Java Class

Name the class: HelloController

STEP 4: Write Controller Code

Add the following code in **HelloController.java**

```
package com.example.service1.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String helloService() {
        return "Hello from Service 1";
    }
}
```

STEP 5 : Change Port of Service 1

Open: application.properties

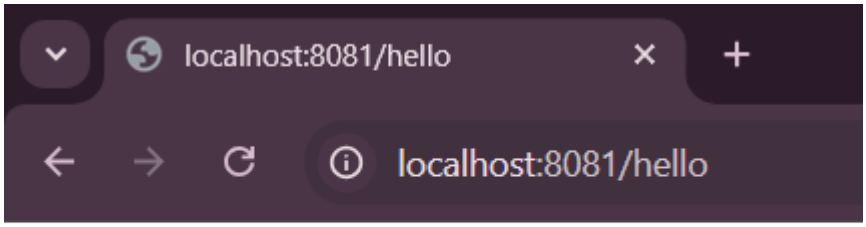
Add: server.port=8081

STEP 6: Run Service 1

Run the application.

Test in browser: http://localhost:8081/hello

Output: Hello from Service 1

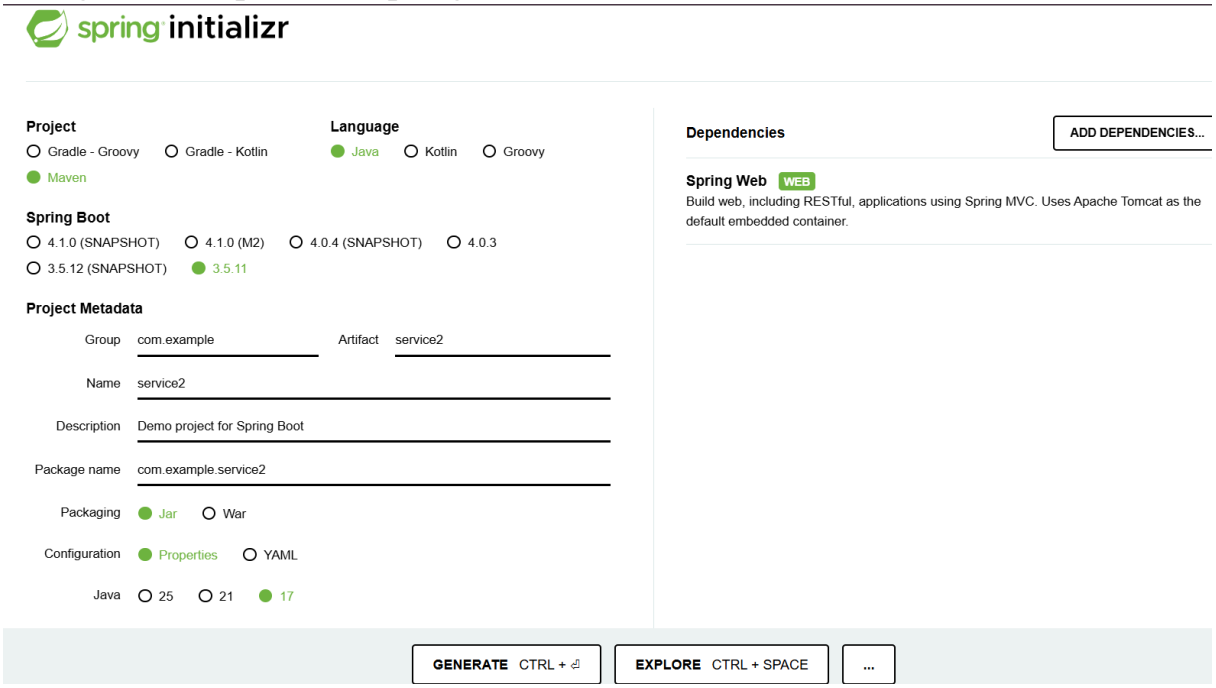


Hello from Service 1

PART II

STEP 1: Create Service 2

Go again to: https://start.spring.io



STEP 2: Change Port of Service 2

Open: application.properties

Add: server.port=8082

STEP 2: Create Controller in Service2

Right click: com.example.service2

Select: New → Package

Name the package: controller

STEP 3: Create Controller Class

Right click on controller package

Select: New → Java Class

Name the class: CallServiceController

```
package com.example.service2.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
@RestController
public class CallServiceController {
    @GetMapping("/call")
    public String callService1() {
        RestTemplate restTemplate = new RestTemplate();
        String response = restTemplate.getForObject(
                "http://localhost:8081/hello",
                String.class
        );
        return "Response from Service1: " + response;
    }
}
```

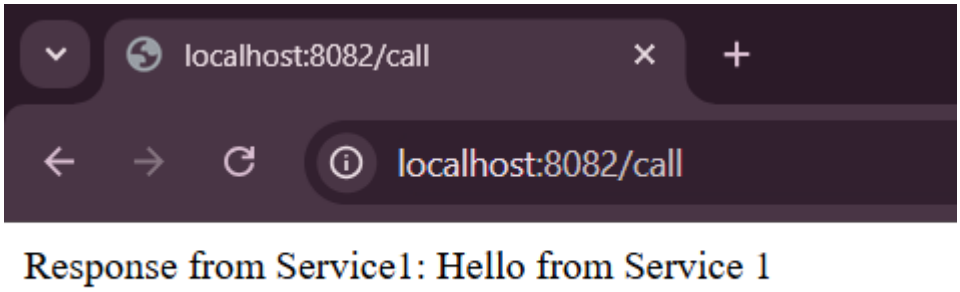
STEP 4: Run Service 2

Service2Application.java

STEP 5: Test Communication

Open browser: http://localhost:8082/call

Output: Response from Service1: Hello from Service 1



Working Flow

Client → Service2 → Service1 → Response

Practical : 7

Aim: Implementing Service Registry- Service Discovery using Eureka

We will create 3 applications

1. Eureka Server

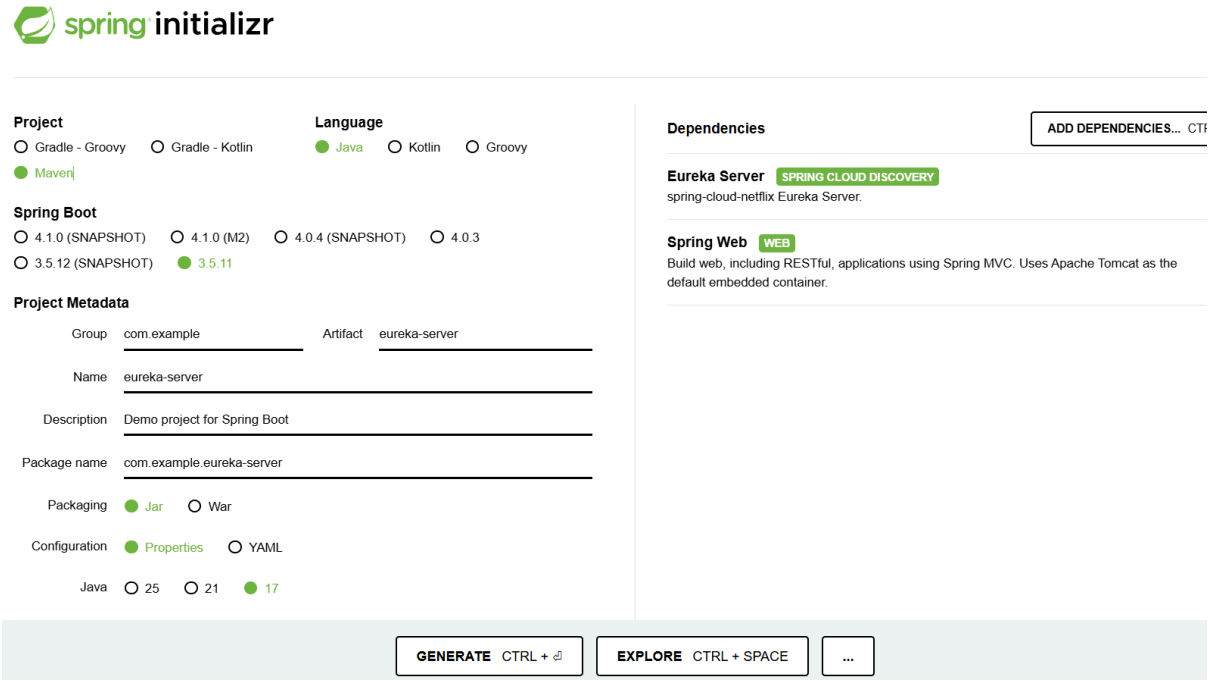
2. Student Service

3. Course Service

PART I

STEP 1: Create Eureka Server

https://start.spring.io



STEP 2: Enable Eureka Server

Open:

EurekaServerApplication.java

Add annotation:

```
package com.example.eureka_server;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
```

```
org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

STEP 3 : Configure application.properties

Open:

src/main/resources/application.properties

Add:

server.port=8761

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

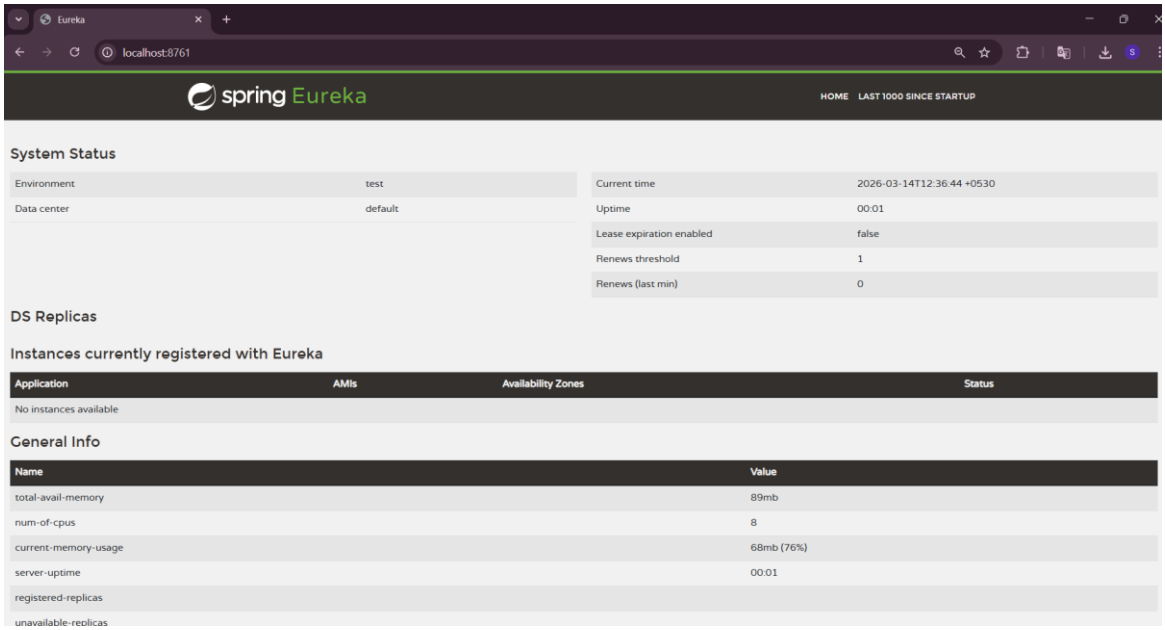
STEP 4 Run Eureka Server

Run the application.

Open browser: http://localhost:8761

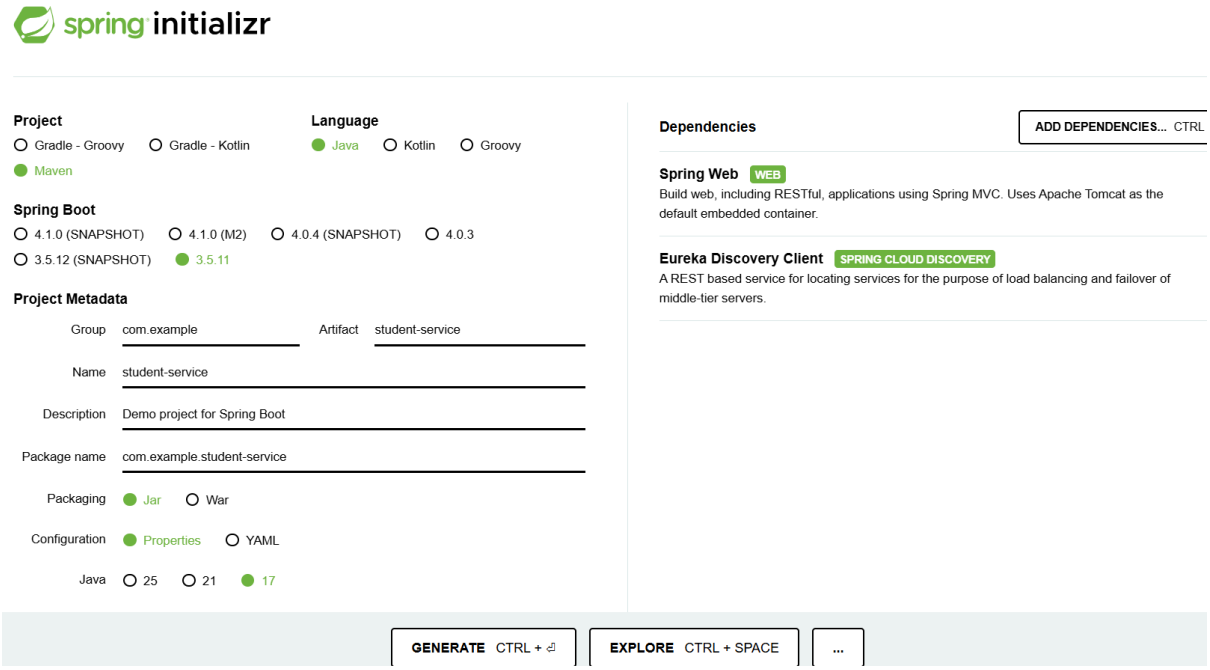
Currently:

No instances available



PART II

STEP 1 — Create Student Service



Click on generate and open project in IntellJ IDE

STEP 2: Configure Properties

Open **application.properties**

Add:

```
spring.application.name=student-service
server.port=8081
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

STEP 3 : Create Student Controller

Create package: controller

Create file under controller package

StudentController.java

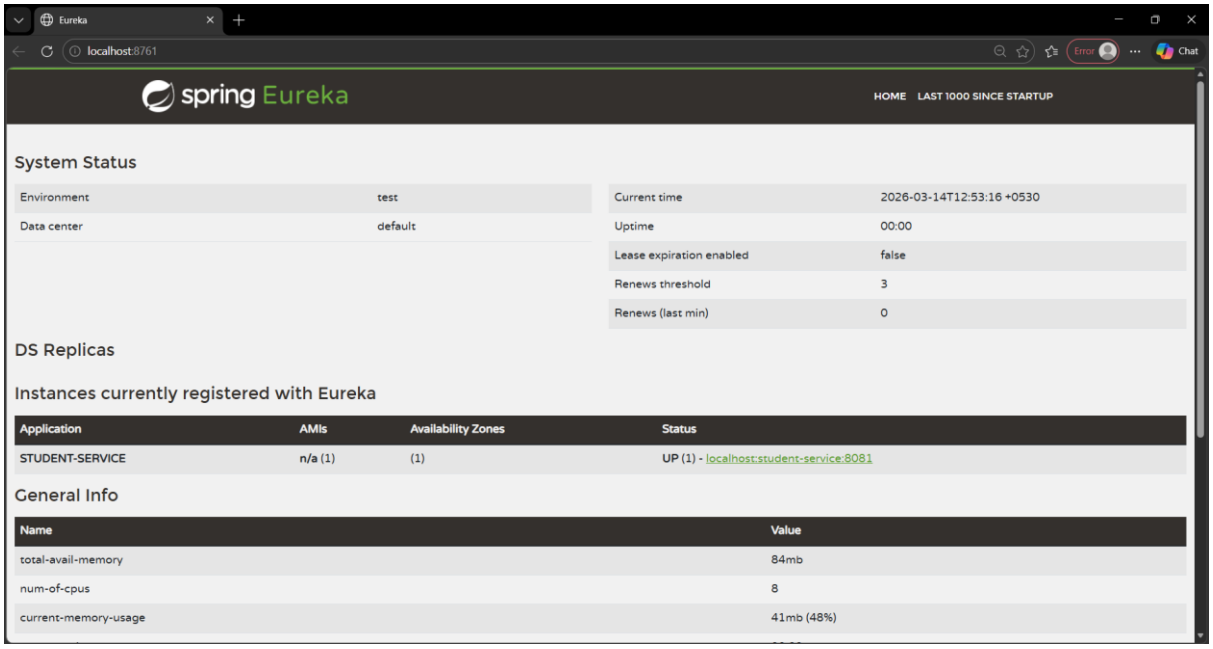
```
package com.example.student_service.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class StudentController {
    @GetMapping("/student")
    public String getStudent() {
        return "Student Service Running";
    }
}
```

STEP 4 : Run Student Service

Run the application.

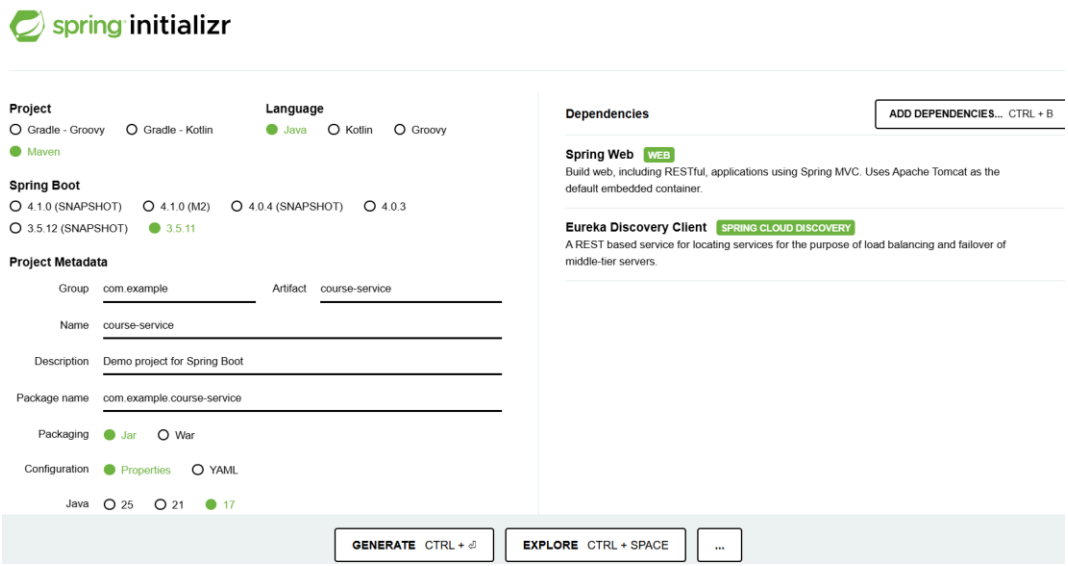
Open: http://localhost:8761



Now you will see: STUDENT-SERVICE Registered.

PART III

STEP 1: Create Course Service



Click on generate and open project in IntellJ IDE

STEP 2: Configure Properties

Open: application.properties

spring.application.name=course-service s

erver.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

STEP 3: Create Controller

Create: CourseController.java in controller package

CourseController.java

```
package com.example.course_service.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class CourseController {
    @GetMapping("/course")
    public String getCourse() {
```

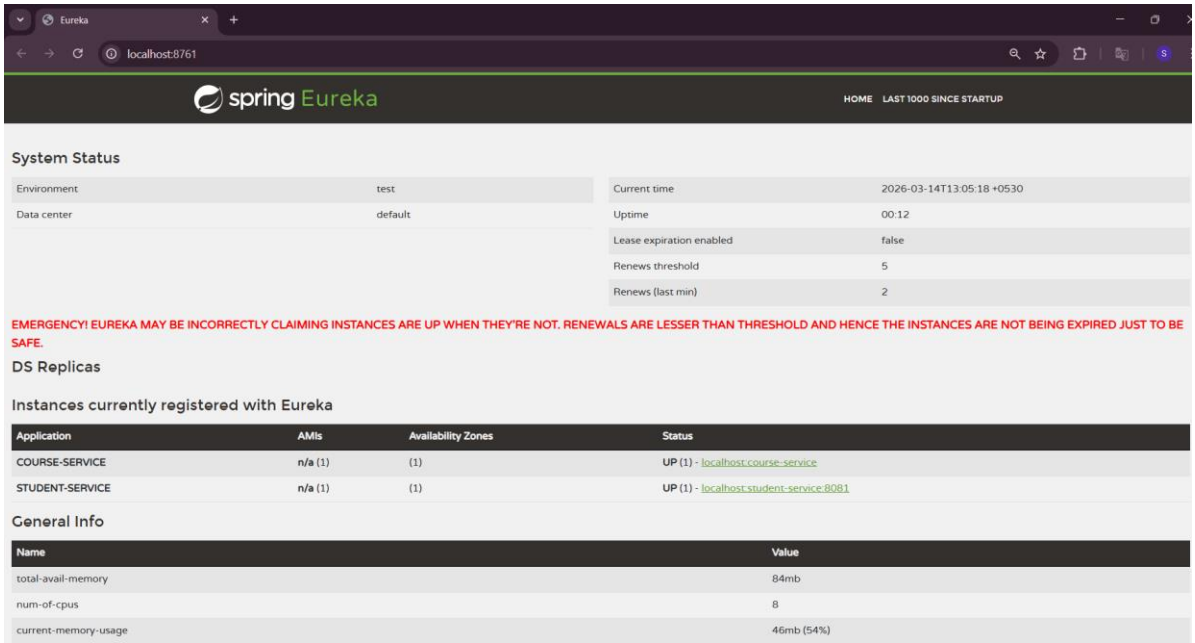
```
        return "Course Service Running";
    }
}
```

STEP 4: Run Course Service

Now open: http://localhost:8761

You will see:

STUDENT-SERVICE

COURSE-SERVICE

Registered successfully.